Vulnerability Assessment Interim Report v1.0

for

GALAXIA RESORTS LLP - SG6399

For

Audit Status	Non-Compliant
Engagement Start Date	6-November-2026
Interim Report Release Date	10-November-2026
Interim Document Version	v1.0
Go-Ahead Report Release Date	NA
Location	Mumbai
Assessment Conducted By	Isha Wagh

Payment Gateway Vulnerability Assessment and Security Testing

Report Created By	Report Created For
CYRAAC Services Private Limited [CyRAACS]	HDFC Bank Ltd.

Confidential Information

The following report contains company confidential information. Do not distribute, email, fax, or transfer via any electronic mechanism unless it has been approved by the recipient company's security policy. All copies and backups of this document should be saved on protected storage always. Do not share any of the information contained within this report with anyone unless they are authorized to view the information. The specific IP addresses / Domain were identified by Client. Our subsequent test work, study of issues in detail and developing action plans are directed towards the issues identified. Consequently, this report may not necessarily comment on all the weaknesses perceived as important by the Client and / or Client management.

Table of Contents

Introduction 4
Scope of Work 4
Engagement Details 4
Auditor Details 4
Report Analysis and Compliance 5
Methodology 5
Overall Compliance Status 6
Assessment Phases 6
Vulnerability Status 7
Vulnerability Ageing 7
Details of Vulnerabilities 8
1. Parameter Manipulation – Request Amount Tampering 8

Introduction

Scope of Work

The project scope covered Vulnerability Assessment and Security Testing for the Payment Gateways integrations for **HDFC Bank**. The vulnerability information details of security testing are as given below:

Engagement Details

Merchant Details	**Name**	GALAXIA RESORTS LLP
	Type	Retail / Service Providers
Ticket ID(TID)		SG6399
Type of Application		Web
Application Details	**Name/URL**	https://www.galaxiaresorts.com/testpayment
	Version Number	NA
	Package Name	NA
	Programming Language	Backend: Node.js (TypeScript) / Express.js Frontend: Next.js 14 / React
Type of Payment Gateway		VAS
Name of Payment Gateway		Smartgateway
Reference/Standards		RBI Advisory (1.4) and OWASP Top 10
Testing Approach	**Type of Environment**	UAT
	Methodology	End to End
	Mode of Execution	Internet
	Mode of Payment	NetBanking
SPOC Detail		PG HelpDesk Team

Auditor Details

Audit Conducted By	**Isha Wagh**
Audit Reviewed By	**CyRAAC Services Private Limited**
Audit Start Date	6-November-2026
Final Go-Ahead Report Release Date	NA

Report Analysis and Compliance

The issues identified and the proposed action plans in this report are based on testing conducted by the VAPT team of CyRAACS. Efforts have been made to verify the accuracy and authenticity of the gathered information where deemed necessary.

The identification of the issues in the report is primarily based on the tests carried out during the limited time for conducting such an exercise.

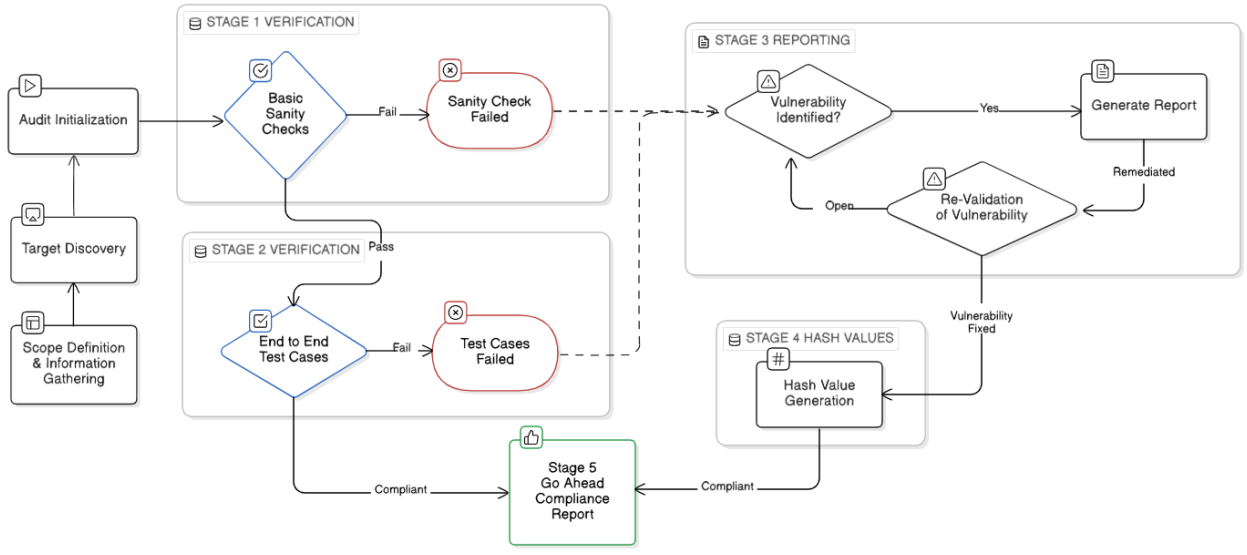
This report is valid as of NA. Any vulnerability, which may have been discovered after this or any exploit, been made available after the above stated date, does not come under the purview of this report.

Any configuration changes or software/hardware updates made on hosts/machines on the IT Infrastructure covered in this test after the date mentioned herein may impact the security posture either positively or negatively and hence invalidates the claims & observations in this report. Whenever there is an update on the IT Infrastructure, we recommend that you conduct a penetration test to ensure that your security posture complies with your security policies.

Methodology

The assessment was conducted in multiple phases, during which the application under test was systematically analyzed to identify potential security vulnerabilities.

The steps outlined below provide a high-level overview of the methodology employed, while the accompanying flowchart illustrates the overall process and sequence followed throughout the assessment.



Overall Compliance Status

This section summarizes the compliance status and the number of new vulnerabilities identified during each phase of assessment.

Compliance Status		**Non-Compliant**					
Assessment Phase		Initial Assessment	Re-Validation - 1	Re-Validation - 2	Re-Validation - 3	Re-Validation - 4	Re-Validation - 5
Total Number of New Vulnerabilities Identified		1	0	0	0	0	0
Request URL	**URL**	https://www.galaxiaresorts.com/api/payments/hdfc-test/create-order					
	Hash Value						
Response URL	**URL**	https://galaxiaresorts.com/testpayment/hdfc-return					
	Hash Value						

Assessment Phases

This section outlines the validation timelines and ISG approval status for the merchant re-validation. Each phase is tracked with corresponding dates and approval updates where applicable.

Merchant Name		GALAXIA RESORTS LLP			
Interim Report Status	**Initial Assessment**	10-November-2026	Document Version	**v1.0**	

Vulnerability Status

Sr No	Severity	Vulnerability Name	Validation Status	Remark
1	HIGH	Parameter Manipulation – Request Amount Tampering	Open	Found in Initial Assessment

Vulnerability Ageing

This section provides detailed information on each identified vulnerability, including its risk level, re-validation status, ageing based on re-validation rounds, and any relevant remarks on remediation progress.

Sr. No	Found in Phase	Validation Status				
		Vulnerability Name	Risk	Fixed/ Open	Reported Date	Fixed Date
1	Initial Assessment	Parameter Manipulation – Request Amount Tampering	HIGH	Open	10-November-2026	NA

Details of Vulnerabilities

HIGH

1. Parameter Manipulation – Request Amount Tampering

Summary:

During transaction processing, the backend does not recompute or validate the amount against server-side authoritative data (e.g., product price, order total stored in database). Instead, it directly trusts the user-supplied value and forwards it to the payment gateway or uses it for transaction processing.

As the parameters are not bound to the session, order ID, or server-side state, an attacker can intercept and modify the request in transit (e.g., via Burp Suite Proxy/Repeater) and submit arbitrary values. The application includes a client-controlled amount parameter in the payment initiation request (via GET/POST/JSON). This parameter is transmitted in plaintext or weakly encoded (e.g., Base64), without any cryptographic integrity protection (HMAC, checksum, digital signature, or token binding).

The parameter manipulation is caused between the checkout page and Payment Gateway Page before performing payment Transaction. An adversary could edit the parameter passing in plain text in request part. In which a much lower amount is paid for a product/service whose initial cost was a higher amount.

Risk:

An attacker can exploit the unvalidated amount parameter passed in plain text between the checkout page and the payment gateway. By modifying the request to reflect a lower amount, the attacker can complete purchases of high-value products or services at significantly reduced costs. This can lead to severe financial losses, inaccurate transaction records, and loss of business trust. Repeated exploitation can cause systemic fraud, impacting both customer experience and merchant reputation.

Business logic vulnerability caused by improper validation and lack of integrity allows attackers to exploit and manipulate transaction values with minimal effort, leading to severe financial impact.

Impact:

An attacker can intercept and tamper with the client-supplied amount parameter before it reaches the payment gateway, causing the backend to process manipulated transaction values and resulting in unauthorized underpayments and potential large-scale financial loss, due to the absence of server-side validation and cryptographic integrity controls over critical payment parameters.

References

OWASP: A01:2025 Broken Access Control

https://cwe.mitre.org/data/definitions/472.html
https://owasp.org/www-community/attacks/Web_Parameter_Tampering
https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

Steps to Reproduce:

- Intercept Payment Request: Configure an intercepting proxy (e.g., Burp Suite) and navigate to the application's checkout/payment page. Initiate a transaction and capture the outgoing payment initiation request.
- Locate amount Parameter: Analyze the intercepted request and identify the amount parameter in the request body (JSON/POST) or URL query string. Verify whether the value is in plaintext or easily decodable format (e.g., Base64).
- Modify the Parameter: Change the amount value to a lower value (e.g., "amount": "5000" → "amount": "5") and forward the request or send it via Repeater.
- Forward Tampered Request: Submit the modified request to the server/payment endpoint and proceed with the payment flow.
- Observe Payment Gateway Behavior: Verify whether the payment gateway or checkout page reflects the manipulated amount instead of the original value.
- Confirm Transaction Integrity: Complete the transaction and check the final confirmation/receipt page to confirm whether the system processes and records the tampered amount instead of the actual price.

Remediations:

- Enforce Server-Side Price Calculation: Never trust the client-supplied amount. Compute the final payable amount on the server using authoritative data (product price, quantity, taxes, discounts) stored in the backend/database. Ignore or discard the amount parameter received from the client.
- Bind Transaction Data to Order Context: Associate each transaction with a server-generated order_id and store the calculated amount against it. During payment initiation and confirmation, validate that the amount matches the value mapped to the corresponding order_id and authenticated user session.
- Implement Message Integrity Protection: Protect sensitive parameters using a cryptographic mechanism such as HMAC (e.g., HMAC-SHA256). Generate a signature on the server using a secret key: "signature = HMAC(secret, order_id + amount + timestamp)"; Validate this signature on receipt to detect any tampering of the amount or related parameters.
- Use Tokenization / Server-Generated Payment Sessions: Replace raw parameters with a server-generated payment token or session ID. The client should only send this token, while all critical values (amount, currency, order details) are retrieved securely on the server side during processing.
- Validate at Payment Gateway & Callback Stage: Ensure the payment gateway request uses server-calculated values. On receiving the gateway callback/webhook, re-verify the transaction by comparing the paid amount with the stored order amount before marking the transaction as successful.
- Implement Input Validation, Logging & Alerts: Reject any request where the amount deviates from the expected server-side value. Log all mismatches and suspicious modifications, and trigger alerts for repeated tampering attempts to detect and prevent automated abuse.

Vulnerable URL(s): https://www.galaxiaresorts.com/api/payments/hdfc-test/create-order